Zakura RPC Unauthenticated Exposure and Unbounded Resource Consumption Vulnerability Report

Summary

Zakura permits RPC to listen beyond loopback while cookie authentication is disabled. In that deployment, any reachable user can invoke expensive methods with very large ranges or address lists, forcing long scans and large result collection that consume node CPU, memory, and disk I/O.

Mitigating Factors:

- Cookie authentication is enabled by default, and a loopback-only listener prevents anonymous remote access.
- The impact depends on chain and index size; an empty test chain demonstrates missing limits but not maximum production impact.
- The separate sendrawtransaction retry-channel issue is not counted again in this report.
- Network firewalls and an authenticated reverse proxy can reduce reachability while a code fix is prepared.

Aggravating Factors:

- Configuration loading does not reject or strongly warn about the combination of a non-loopback listener and disabled cookie authentication.
- getnetworksolps accepts extremely large positive num_blocks values, and address methods lack practical item, range, and pagination limits.
- Expensive RPC work shares the same process and resources as peer-to-peer, state, and mempool services.
- Small remote requests can trigger much larger backend work and can be issued concurrently.

Vulnerability Details:

Status, Product, and Version

- Record status: PUBLISHED
- Product and version: Zakura / zakurad / 1.2.0
- Weakness classification: CWE-400: Uncontrolled Resource Consumption; CWE-770: Allocation of Resources Without Limits or Throttling; CWE-306: Missing Authentication for Critical Function

Cross-Validation Conclusion

The configuration and missing resource boundaries form a valid exposure-dependent risk. Default authentication substantially limits the attacker population, but a remotely reachable listener with cookie authentication disabled turns administrative, high-cost methods into anonymous endpoints.

Trigger Conditions

- RPC listens on a non-loopback address.
- Cookie authentication is disabled.
- The attacker can reach the RPC port.
- The attacker repeatedly supplies very large scan ranges or address collections.

Normal and Abnormal Behavior

Normal behavior

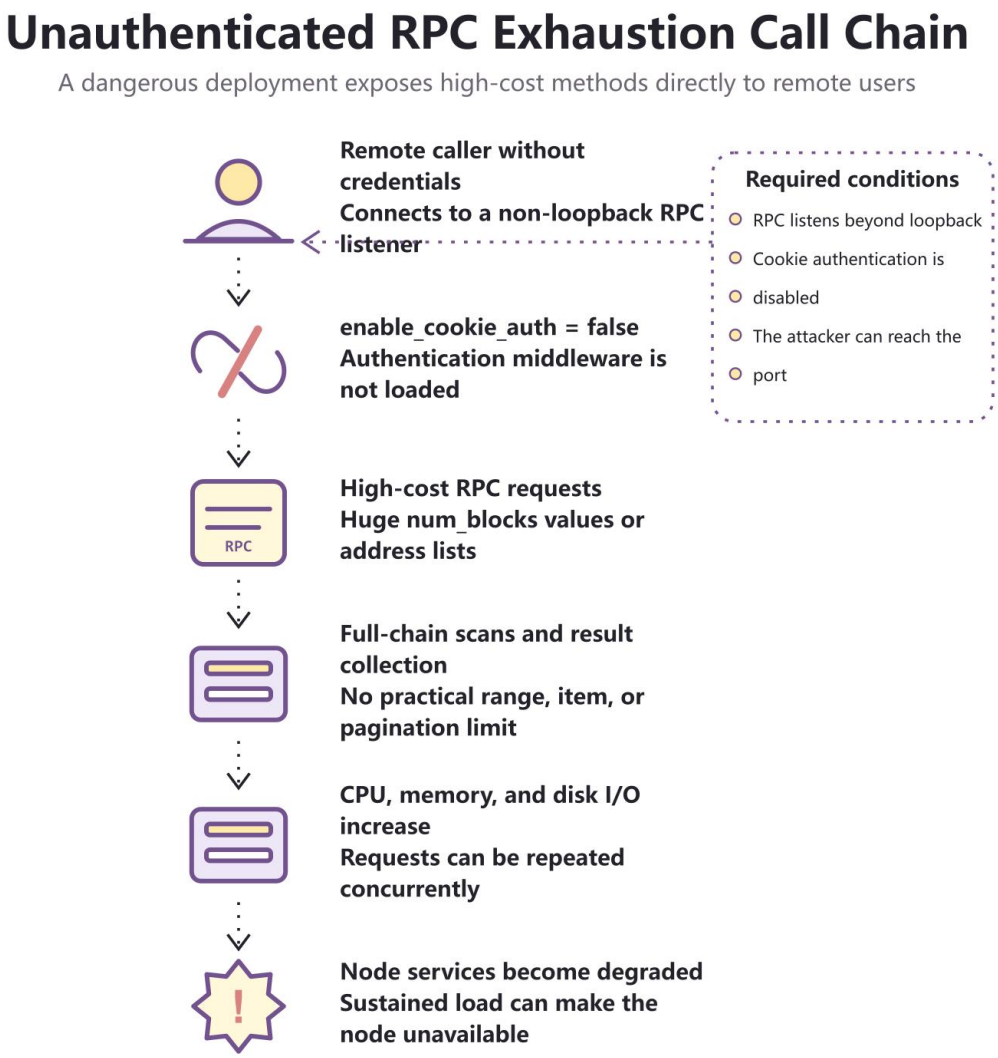
Administrative RPC is authenticated, restricted to trusted networks, and applies explicit work, range, result-size, and concurrency limits.

Abnormal behavior

An anonymous caller can request work far larger than the request itself, while multiple calls compete with core node services for the same resources.

Full Call Chain Diagram

The complete call chain is shown below. The dashed box states the deployment condition, version boundary, or evidence boundary that controls the conclusion.



Technical Details:

Root cause 1: the dangerous deployment combination is accepted

Authentication middleware is loaded only when enable_cookie_auth is true. Configuration validation does not block a non-loopback listener when that switch is false.

```
if config.enable_cookie_auth {
    router = router.layer(cookie_auth_layer(cookie_path));
}
// No rejection for non-loopback + authentication disabled.
```

Root cause 2: history scanning lacks a practical upper bound

getnetworksolps converts a large positive num_blocks value into a traversal count without clamping it to a safe window.

```
let num_blocks = if num_blocks < 1 { DEFAULT } else { num_blocks as usize };
headers.iter().rev().take(num_blocks.checked_add(1).unwrap_or(num_blocks));
```

Root cause 3: address queries can scan and collect too much

Address vectors, height spans, and returned rows are not governed by a single resource budget or pagination contract.

```
let start = request.start.unwrap_or(0);
let end = request.end.filter(|h| *h != 0).unwrap_or(chain_height);
let results: Vec<_> = query_range(start..=end).collect();
```

Minimal Reproduction Logic (Isolated Test Environment Only)

On an isolated node configured with a non-loopback listener and cookie authentication disabled, compare bounded requests with very large num_blocks values and large address arrays while measuring request duration and resource use.

```
{"jsonrpc":"2.0","id":1,"method":"getnetworksolps","params":[2000000000]}
{"jsonrpc":"2.0","id":2,"method":"getaddresstxids","params":[{"addresses":["<add
ress>"],"start":0,"end":0}]}
```

Security Impact

- Sustained requests can degrade RPC responsiveness and compete with synchronization, peer, and mempool work.
- In an exposed deployment, the attacker needs no account or RPC cookie.
- The reviewed evidence supports resource-exhaustion risk; it does not by itself establish fund theft or consensus corruption.

Long-Term Remediation

- Reject startup, or require an explicit high-risk override, when RPC is non-loopback and cookie authentication is disabled.
- Clamp history ranges and num_blocks to documented safe limits.
- Limit address count, height span, result size, and concurrent expensive requests; add pagination.
- Keep RPC on loopback or an authenticated private network and update examples to show the safe default.

Recommended Regression Tests:

- Verify startup behavior for every listener and authentication combination, including IPv4 and IPv6 wildcard addresses.
- Confirm that huge num_blocks values return a controlled error without long traversal.
- Confirm address-count, range, result-size, and concurrency limits.
- Load-test bounded calls while peer synchronization and mempool processing remain responsive.